

تمرین تحلیل کد Mnist.py

```
7
8 import os
9 os.environ['SCIKIT_LEARN_DATA'] = './mnist_data'
10
11 from sklearn.datasets import fetch_openml
12 X, y = fetch_openml('mnist_784', version=1, return_X_y=True, parser='auto')
13 X = X.values
14 y = y.astype(int).values
15
16 print(X.shape)
17 # (70000, 784)
18 print(y.shape)
19 # (70000,)
20
21 X = ((X / 255.) - .5) * 2
22
```

خط های ۸ و ۹ :

متغیر محیطی SCIKIT_LEARN_DATA برابر با آدرس یک دایرکتوری تعریف می‌شود. وقتی scikit-learn دیتاست را دانلود کرد متغیر محیطی را چک کرده و در این آدرس قرار میدهد. بعد از بار اول هر بار کد اجرا شد نگاه میکند که ببیند که آیا در این آدرس دیتاست وجود دارد یا خیر.

خط های ۱۱ تا ۱۴:

با متد fetch_openml از OpenML دیتاست mnist 28*28 (28*28=784) دانلود می‌کند. خروجی آن یک tuple است (به صورت (X,y)) که در آن X از نوع DataFrame و y از نوع Series است.

در خط ۱۳ مقدار X رو در X میگذاریم که با این کار X تبدیل به یک numpy.array می‌شود که از نوع int است. همین‌طور در خط ۱۴ مقادیر y را تبدیل به int میکنیم.

خط ۲۱:

X را که مقادیرش در بازه [0-255] هست را تبدیل به یک ماتریس با مقدار بین [-1,1] می‌کنیم.

```
23 import matplotlib.pyplot as plt
24 fig, ax = plt.subplots(nrows=2, ncols=5, sharex=True, sharey=True)
25 ax = ax.flatten()
26 for i in range(10):
27     img = X[y == i][0].reshape(28, 28)
28     ax[i].imshow(img, cmap='Greys')
29 ax[0].set_xticks([])
30 ax[0].set_yticks([])
31 plt.tight_layout()
32 plt.show()
33
34 fig, ax = plt.subplots(nrows=5, ncols=5, sharex=True, sharey=True)
35 ax = ax.flatten()
36
37 for i in range(25):
38     img = X[y == 7][i].reshape(28, 28)
39     ax[i].imshow(img, cmap='Greys')
40 ax[0].set_xticks([])
41 ax[0].set_yticks([])
42 plt.tight_layout()
43 plt.show()
44
```

خط‌های ۲۳ تا ۳۲:

در خط ۲۴ یک figure درست میکنیم که تصاویر در داخل آن در قالب یک آرایه ۲*۵ نمایش داده شود. ax آرایه است که تصاویر در داخل آن قرار است نمایش داده شوند.

در خط ۲۵ برای راحتی کار در assign کردن تصاویر به هر یک از درآیه های آرایه تصویر ، آرایه را به یک بعدی تبدیل میکنیم.

خط ۲۶ تا ۲۸ هر یک از برچسب‌های ۰ تا ۹ را در X پیدا کرده و اولی را با reshape به اندازه ۲۸*۲۸ در آرایه ax نمایش می‌دهد.

خط ۲۹ و ۳۰ اعدادی که روی محور X و y نمایش داده می‌شود را پاک میکند. ضمناً انتخاب ax[0] برای اعمال این تنظیمات این است که اگر این تنظیمات روی فقط همین المنت اعمال شود روی بقیه subplot ها هم اعمال می‌شود.

خط ۳۱ و ۳۲ هم فاصله بین subplot ها را بهینه کرده و در نهایت شکل را نشان می‌دهد.

خط‌های ۳۴ تا ۴۳:

در این بخش تعداد ۲۵ تا از انواع تصویر با برچسب شماره ۷ در یک figure با 5*5 subplot نمایش داده میشود.

```
45 from sklearn.model_selection import train_test_split
46 X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=10000, random_state=123, stratify=y)
47 X_train, X_valid, y_train, y_valid = train_test_split(X_temp, y_temp, test_size=5000, random_state=123, stratify=y_temp)
48
49 # from neuralnet import NeuralNetMLP
50
51 import numpy as np
52 def sigmoid(z):
53     return 1. / (1. + np.exp(-z))
54 def int_to_onehot(y, num_labels):
55     ary = np.zeros((y.shape[0], num_labels))
56     for i, val in enumerate(y):
57         ary[i, val] = 1
58     return ary
59
```

خط‌های ۴۵ تا ۵۸:

در این بخش داده‌ها به ۳ بخش `test` , `train` و `validation` تقسیم می‌شوند. در خط ۴۵ ابتدا تعداد ۱۰۰۰۰ عدد از دیتاست برای تست جدا می‌شود. `random_state` برای این است که هر بار این خط کد اجرا شود داده‌های انتخاب شده یکسان انتخاب شوند. `startify` بر اساس توزیع برچسب‌ها در دیتاست داده‌ها را تقسیم میکند تا نسبت برچسب‌ها در هر یک از بخش‌های `test` , `train` و `validation` مانند نسبت برچسب‌ها در دیتاست اصلی باشد.

خط‌های ۵۱ تا ۵۸:

تابع `sigmoid` در خط ۵۱ تا ۵۳ تعریف شده است.

تابع `int_to_onehot` هم در خط ۵۴ تا ۵۸ لیبل‌های دیتاست که در `Y` است را به یک ماتریس با تعداد سطرهای خود `Y` و تعداد ستونهایی برابر با تعداد نوع‌های برچسب‌های موجود در `Y` تبدیل میکند. انواع برچسب‌هایی که داریم ۰ تا ۹ است. پس اگر برای مثال یکی از برچسب‌ها در `Y` برابر ۸ باشد در آن سطر تمام درآیه‌ها برابر صفر بود و فقط ستون ۸ام برابر ۱ می‌شود.

```
60
61 class NeuralNetMLP:
62     def __init__(self, num_features, num_hidden,num_classes, random_seed=123):
63         super().__init__()
64         self.num_classes = num_classes
65         # hidden
66         rng = np.random.RandomState(random_seed)
67         self.weight_h = rng.normal(
68             loc=0.0, scale=0.1, size=(num_hidden, num_features))
69         # loc:float or array_like of floats
70         # /Mean ("centre") of the distribution.
71         # scale:float or array_like of floats
72         # /Standard deviation (spread or "width") of the distribution. Must be non-negative.
73         self.bias_h = np.zeros(num_hidden)
74         # output
75         self.weight_out = rng.normal(
76             loc=0.0, scale=0.1, size=(num_classes, num_hidden))
77         self.bias_out = np.zeros(num_classes)
78
```

خط‌های ۶۱ تا 139:

در اینجا کلاس `NeuralNetMLP` عملیات آموزش شبکه عصبی تعریف شده است. این کلاس شامل ۳ متد است. متد `init` برای مقداردهی اولیه `member variable` های کلاس، متد `forward` برای تعریف عملیات `feed forward` و متد `backward` برای تعریف عملیات `backpropagation`.
خط 62 تا 77 `constructor` کلاس است که مقادیر اولیه در زمان ایجاد شی را در `member variable` های شی قرار میدهد.
خط 63 برای این است که اگر این کلاس فرزند کلاس دیگری باشد در این مرحله ابتدا کلاس والد `construct` شود.
خط 66 تا 77 برای `weight` ها و `bias` ها مقدار اولیه می‌سازد.
خط 66 یک `random generator` می‌سازد که آرگومان `random_seed` به عنوان ورودی به تابع سازنده آن داده شده است. با این کار هر بار این کد را اجرا کنیم همان مقادیر تصادفی برای `weight` توسط `generator` قرار داده می‌شود.
خط 67 و 75 با استفاده از توزیع نرمال و با میانگین 0 و واریانس 0/1 وزن‌هایی با سائزی که در آن مشخص شده است را به طور `random` ایجاد میکند.
خط 73 و 77 یک ماتریس صفر به عنوان مقدار اولیه بایاس‌ها می‌سازد.

```
79     def forward(self, x):
80         # Hidden layer
81         # input dim: [n_hidden, n_features]
82         # dot [n_features, n_examples].T
83         # output dim: [n_examples, n_hidden]
84         z_h = np.dot(x, self.weight_h.T) + self.bias_h
85         a_h = sigmoid(z_h)
86         # Output layer
87         # input dim: [n_classes, n_hidden]
88         # dot [n_hidden, n_examples].T
89         # output dim: [n_examples, n_classes]
90         z_out = np.dot(a_h, self.weight_out.T) + self.bias_out
91         a_out = sigmoid(z_out)
92         return a_h, a_out
```

خط 79 تا 92:

متد `forward` مسیر رو به جلوی شبکه را تعریف میکند.

خط 84 و 85 مجموع اعمال انجام شده در لایه `hidden` است. ابتدا ورودی در `Weight_h` ضرب شده و بعد با `bias_h` جمع می‌شود. در مرحله بعد تابع فعال‌سازی `sigmoid` روی خروجی اعمال می‌شود و خروجی را در محدوده 0 تا 1 قرار می‌دهد. برای لایه `output` هم به همین صورت عمل می‌شود.
با تعریف فاکتورهای زیر ، `shape` ماتریس‌های این مرحله در جدول نمایش داده می‌شود:

تعداد بچ ها **B** , تعداد نورون های لایه **H = hidden** , تعداد کلاس ها **C** , تعداد پیکسل های ورودی (28 در 28 برابر با 784) **F**

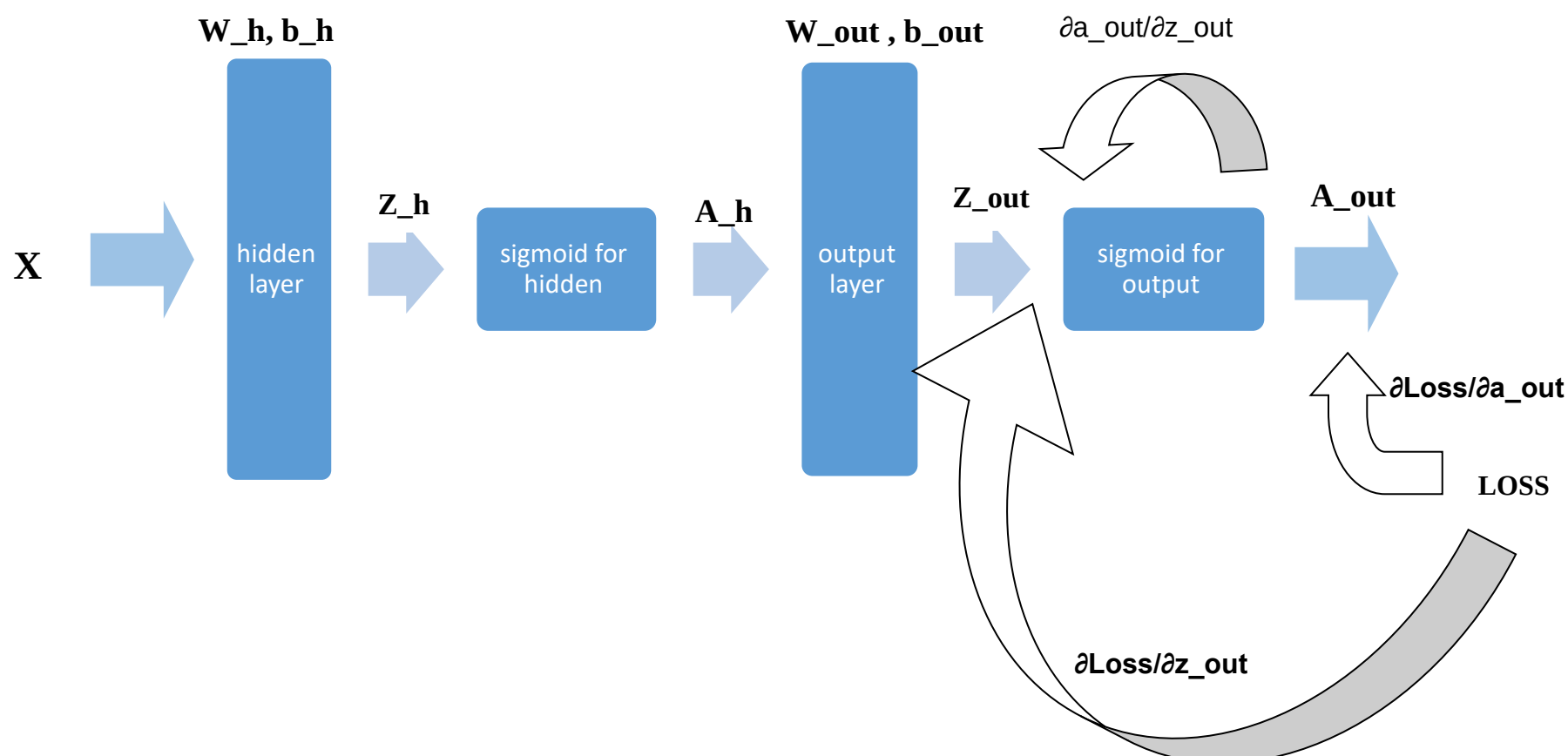
ماتریس	Shape
X	(B,F)
Weight_h	(H,F)
Bias_h	(H,)
a_h و Z_h	(B,H)
weight_out	(C,H)
Bias_out	(C,)
a_out و Z_out	(B,C)

```
95 def backward(self, x, a_h, a_out, y):
96     #####
97     ### Output layer weights
98     #####
99     # one-hot encoding
100     y_onehot = int_to_onehot(y, self.num_classes)
101     # y_onehot = int_to_onehot(y, n_classes)
102     # Part 1: dLoss/dOutWeights
103     ## = dLoss/dOutAct * dOutAct/dOutNet * dOutNet/dOutWeight
104     ## where DeltaOut = dLoss/dOutAct * dOutAct/dOutNet
105     ## for convenient re-use
106     # input/output dim: [n_examples, n_classes]
107     d_loss__d_a_out = 2.*(a_out - y_onehot) / y.shape[0]
108     # input/output dim: [n_examples, n_classes]
109     d_a_out__d_z_out = a_out * (1. - a_out) # sigmoid derivative
110     # output dim: [n_examples, n_classes]
111     delta_out = d_loss__d_a_out * d_a_out__d_z_out
112     # gradient for output weights
113     # [n_examples, n_hidden]
114     d_z_out__dw_out = a_h
115     # input dim: [n_classes, n_examples]
116     # dot [n_examples, n_hidden]
117     # output dim: [n_classes, n_hidden]
118     d_loss__dw_out = np.dot(delta_out.T, d_z_out__dw_out)
119     d_loss__db_out = np.sum(delta_out, axis=0)
120
121     #####
122     # Part 2: dLoss/dHiddenWeights
123     ## = DeltaOut * dOutNet/dHiddenAct * dHiddenAct/dHiddenNet
124     # * dHiddenNet/dWeight
125     # [n_classes, n_hidden]
126     # d_z_out__a_h = weight_out
127
128     d_z_out__a_h = self.weight_out
129     # output dim: [n_examples, n_hidden]
130     d_loss__a_h = np.dot(delta_out, d_z_out__a_h)
131     # [n_examples, n_hidden]
132     d_a_h__d_z_h = a_h * (1. - a_h) # sigmoid derivative
133     # [n_examples, n_features]
134     d_z_h__d_w_h = x
135     # output dim: [n_hidden, n_features]
136     d_loss__d_w_h = np.dot((d_loss__a_h * d_a_h__d_z_h).T,
137     d_z_h__d_w_h)
138     d_loss__d_b_h = np.sum((d_loss__a_h * d_a_h__d_z_h), axis=0)
139     return (d_loss__dw_out, d_loss__db_out, d_loss__d_w_h, d_loss__d_b_h)
140
```

خط های 95 تا 139:

متد backward مسیر پس انتشار را تعریف می کند. در اینجا هدف این است که با استفاده از خروجی به دست آمده در مرحله feed forward ، گرادیان خطا نسبت به هر یک از weight ها و biasها را به دست آوریم تا بتوانیم با یافتن میزان سهم هر یک از وزن ها و بایاس ها در خطا ، در مرحله بعد با استفاده از آنها بتوانیم وزن ها را به روز رسانی کرده و با تکرار این چرخه مقدار Loss را به کمترین مقدار ممکن برسانیم.

نحوه به دست آوردن مقادیر گرادیان مد نظر به صورت زیر است:



خط 100 ماتریس y (برچسب های دیتاست) را به ماتریس Onehot تبدیل می کند. در این ماتریس سطر ها به تعداد نمونه های دیتاست و ستون به تعداد کلاس های موجود در دیتاست است. در این ماتریس تمام درآیه ها صفر است به جز درآیه ی مربوط به کلاس نمونه مورد نظر، که برابر 1 است. چون خروجی به دست آمده از forward pass (a_{out}) برای هر نمونه و به ازای هر کلاس یک مقدار بین 0 تا 1 به عنوان احتمال عضویت در آن کلاس در داخل ماتریس قرار داده، ماتریس onehot را به عنوان شاخص به محاسبه خطا از y به دست آوردیم.

خط 107 $\partial \text{Loss} / \partial a_{out}$ را حساب کرده است. فرمول آن از مشتق گیری از فرمول MSE به دست آمده است. $\text{Shape} = (B, C)$

خط 109 $\partial a_{out} / \partial z_{out}$ را محاسبه می کند. از آنجا که a_{out} و z_{out} خروجی و ورودی تابع sigmoid هستند این مقدار برابر است با مشتق sigmoid. $\text{Shape} = (B, C)$

خط 111 برای محاسبه $\partial \text{Loss} / \partial z_{out}$ از قاعده زنجیری استفاده کرده است:

$$\partial \text{Loss} / \partial z_{out} = \partial \text{Loss} / \partial a_{out} * \partial a_{out} / \partial z_{out}$$

$$\text{Shape} = (B, C)$$

خط 114 از آنجا که $z_{out} = a_h * w_{out} + b_{out}$ در نتیجه $\partial z_{out} / \partial w_{out}$ برابر a_h است.

خط 118 طبق قاعده زنجیری $\partial \text{Loss} / \partial z_{out} = \partial z_{out} / \partial w_{out} * \partial \text{Loss} / \partial z_{out}$ که همان عبارت نوشته شده در کد است.

خط 119 چون $z_{out} = a_h * w_{out} + b_{out}$ در نتیجه $\partial \text{Loss} / \partial b_{out}$ برابر با 1 می شود.

خط 128 تا 138 نیز با منطق مرحله قبل پیش می رود.

	Name ▲	Type	Size	Value
	a_h	Array of float64	(100, 50)	[[0.52132303 0.09993542 0.28041434 ... 0.42970783 0.98821984 0.1685391 ...
	a_out	Array of float64	(100, 10)	[[0.66706787 0.55655311 0.51688579 ... 0.42344007 0.61336513 0.5696634 ...
	acc	float64	1	np.float64(0.0936)
	ax	Array of object	(25,)	ndarray object of numpy module
	d_a_h__d_z_h	Array of float64	(100, 50)	[[0.24954533 0.08994833 0.20178214 ... 0.24505901 0.01164139 0.1401336 ...
	d_a_out__d_z_out	Array of float64	(100, 10)	[[0.22208833 0.24680175 0.24971487 ... 0.24413858 0.23714835 0.245147 ...
	d_loss__a_h	Array of float64	(100, 50)	[[1.77317897e-04 1.57315255e-04 -2.96050385e-04 ... -2.09684313e-04 ...
	d_loss__d_a_out	Array of float64	(100, 10)	[[0.01334136 0.01113106 0.01033772 ... 0.0084688 -0.0077327 0. ...
	d_loss__d_b_h	Array of float64	(50,)	[0.00722408 0.00461723 0.00628065 ... 0.00015521 0.00261951 -0.0 ...
	d_loss__d_w_h	Array of float64	(50, 784)	[[-0.00722408 -0.00722408 -0.00722408 ... -0.00722408 -0.00722408 -0 ...
	d_loss__db_out	Array of float64	(10,)	[0.22353069 0.20748106 0.20675237 0.13169432 0.23422251 0.22011355 0. ...
	d_loss__dw_out	Array of float64	(10, 50)	[[0.12664688 0.13294218 0.0669687 ... 0.06059253 0.17231108 0.0714846 ...
	d_z_h__d_w_h	Array of float64	(100, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	d_z_out__a_h	Array of float64	(10, 50)	[[0.20763998 0.06375588 0.00478648 ... 0.02203666 0.06760708 -0 ...
	d_z_out__dw_out	Array of float64	(100, 50)	[[0.52132303 0.09993542 0.28041434 ... 0.42970783 0.98821984 0.1685391 ...
	delta_out	Array of float64	(100, 10)	[[0.00296296 0.00274717 0.00258148 ... 0.00206756 -0.0018338 0. ...
	fig	figure.Figure	1	Figure object of matplotlib.figure module
	i	int	1	0
	img	Array of float64	(28, 28)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	minibatch_gen	generator	1	generator object
	minibatch_size	int	1	100

	Name ▲	Type	Size	Value
	minibatch_size	int	1	100
	model	NeuralNetMLP	1	NeuralNetMLP object of __main__ module
	mse	float64	1	np.float64(0.27625385706671446)
	num_epochs	int	1	50
	predicted_labels	Array of int64	(5000,)	[8 8 8 ... 8 2 8]
	probas	Array of float64	(5000, 10)	[[0.53928539 0.56634271 0.50265227 ... 0.38341305 0.71483183 0.6179647 ...
	self	NeuralNetMLP	1	NeuralNetMLP object of __main__ module
	X	Array of float64	(70000, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	x	Array of float64	(100, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	X_temp	Array of float64	(60000, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	X_test	Array of float64	(10000, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	X_train	Array of float64	(55000, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	X_train_mini	Array of float64	(100, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	X_valid	Array of float64	(5000, 784)	[[-1. -1. -1. ... -1. -1. -1.] [-1. -1. -1. ... -1. -1. -1.]
	y	Array of int64	(100,)	[8 1 9 ... 6 7 1]
	y_onehot	Array of float64	(100, 10)	[[0. 0. 0. ... 0. 1. 0.] [0. 1. 0. ... 0. 0. 0.]
	y_temp	Array of int64	(60000,)	[4 8 7 ... 2 4 3]
	y_test	Array of int64	(10000,)	[5 9 6 ... 2 7 1]
	y_train	Array of int64	(55000,)	[4 5 4 ... 4 3 0]
	y_train_mini	Array of int64	(100,)	[2 1 3 ... 7 2 4]
	y_valid	Array of int64	(5000,)	[5 5 1 ... 2 4 9]

